



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

Advanced AI Phishing Detection System

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Introduction

The purpose of this assignment is to extend an existing Machine Learning-based phishing detection system, improving its ability to identify sophisticated attacks that evade traditional keyword-based methods. The work is organised into six sequential tasks, each addressing a specific technical weakness of the baseline system.

Before any modification, the baseline model (Logistic Regression) was trained on dataset_large.csv (2,500 samples, balanced split 80/20). The initial metrics were very high.

Κλάση	Precision	Recall	F1-Score
Legitimate	0.996	1.000	0.998
Phishing	1.000	0.996	0.998
Overall Accuracy			99.8%

```
(.venv) C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

```
(.venv) C:\Users\Kalai\Desktop\ai_phishing>
```

Despite the high metrics, the baseline model shows weaknesses on borderline cases (polite phishing without urgency language), which motivated the improvements described in Tasks 1 and 2.

```

C:\Users\Kakai>cd Desktop\ai_phishing

C:\Users\Kakai\Desktop\ai_phishing>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 812-306-854
127.0.0.1 - - [14/Jun/2026 01:27:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 01:27:06] "GET /favicon.ico HTTP/1.1" 404 -

```

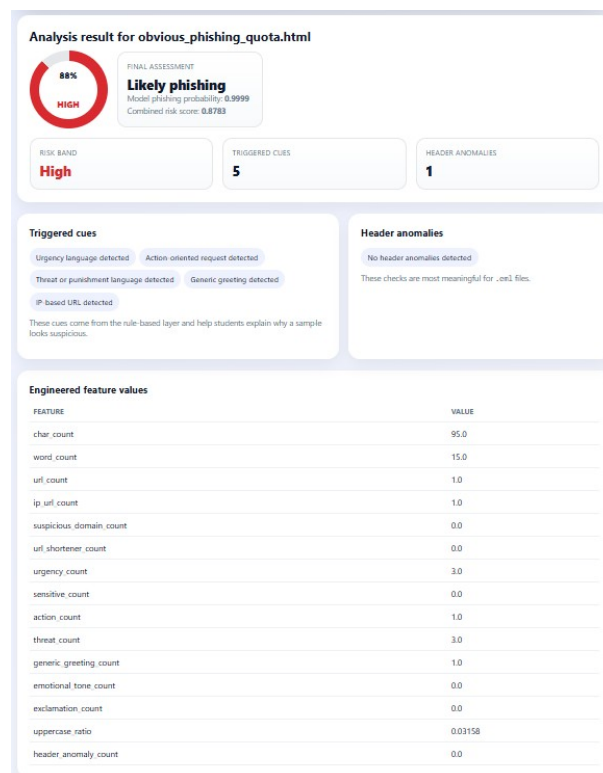
Master's Lab: AI Phishing Detection

Upload an .html, .docx, or .eml file. The app combines a machine-learning model, phishing heuristics, and email-header checks to produce a risk-oriented classroom analysis.

Try the files in `sample_inputs/` and compare obvious, legitimate, and borderline examples.

- The final label and risk score.
- Which cues were triggered.
- Whether header anomalies were found.
- Which feature values look unusually high.

The first sample (`obvious_phishing_quota.html`) is a clear phishing case with an IP-based URL and threatening language. The system correctly classified it as high-risk.



The second sample (`hard_legitimate_portal.html`) is a legitimate email. The system correctly classified it as 'Likely legitimate' with a low Risk Score.

The third sample (hard_phishing_polite.html) reveals the critical gap of the baseline. It is a phishing email written with polite language, without any urgency keywords — precisely the cases that the system's approach fails to detect.

This weakness is the motivation for the improvements described in Tasks 1 and 2.

Task 1: Feature Engineering

The original system extracted features based exclusively on threat and urgency keywords. This approach creates a blind spot for phishing campaigns that use persuasion instead of threats. Four new features were designed to cover four specific attack categories invisible to the baseline:

- a) Phishing using 'bait' instead of threats
- b) Malicious URLs with credential bypass techniques
- c) Sender identity spoofing
- d) Algorithmically generated domains (DGA)

These four gaps correspond to known attack categories analysed in the course (Lecture 5: Fraud, Spam, and Phishing).

Feature 1-reward_bait_count

Purpose: Counts the presence of 'bait' words (e.g. won, prize, gift card, congratulations, exclusive offer). Targets attacks that attract victims with promises of reward rather than threatening them.

Code changes: At the beginning of feature_extraction.py, immediately after the imports, the following pattern list was added:

```
REWARD_BAIT_PATTERNS = [  
    r"\bwon\b", r"\bwinner\b", r"\bprize\b", r"\breward\b",  
    r"\bgift card\b", r"\bcongratulations?\b", r"\bselected\b",  
    r"\bexclusive offer\b", r"\bclaim your\b", r"\bfree access\b",  
    r"\byou have been chosen\b", r"\bspecial deal\b",  
]
```

In the `compute_handcrafted_features()` function, after the URLs for loop, the following calculation was added:

```
# Feature 1: Reward/bait language
reward_bait_count = float(_count_matches(lowered, REWARD_BAIT_PATTERNS))
```

Και στο return dictionary προστέθηκε η νέα τιμή:

```
And the new value was added to the return dictionary:
```

Feature 2-at_symbol_url_count

Purpose: Detects URLs containing the @ symbol (e.g. `http://legit.com@evil.com`). This technique exploits the URL parsing logic of browsers, which ignores everything before @ when resolving the actual host.

Code changes: In the `compute_handcrafted_features()` function, before the for loop, the following initialisation was added:

```
at_symbol_urls = 0 # Task 1 – Feature 2
```

Inside the for loop, immediately after the check for URL shorteners, the following was added:

```
# Feature 2: @ symbol in URL — credential redirect trick
if "@" in url:
    at_symbol_urls += 1
```

Και στο return dictionary:

```
"at_symbol_url_count": float(at_symbol_urls),
```

Purpose: Checks whether the sender's display name contains the name of a known organisation (Microsoft, PayPal, HR, Admin etc.) while the actual sending domain does not match.

Purpose: Calculates the Shannon mathematical entropy of each domain name appearing in the message's URLs, using the `_domain_entropy()` function.

Entropy measures the 'randomness' of a character sequence. Natural domains (e.g. google.com, university.edu) use a limited vocabulary with repeated patterns, resulting in low entropy.

Attackers use DGA domains precisely to evade static blacklists. Instead of using a fixed malicious domain, they generate new ones algorithmically.

Code changes: Implemented via a new `_domain_entropy()` helper function in `feature_extraction.py`:

```
"it support", "hr", "payroll", "helpdesk", "security team",  
"admin", "administrator", "noreply", "no-reply", "support",  
]
```

Inside the for loop of `compute_handcrafted_features()`:

```
# Feature 3: Display-name spoofing  
display_name_spoof = 0.0  
if headers and headers.from_addr:  
    from_raw = headers.from_addr.lower()  
    m = re.match(r'^(.+?)s*<', from_raw)  
    if m:  
        display_name = m.group(1).strip().strip("\"")  
        from_domain_val = extract_domain(headers.from_addr)  
        if any(org in display_name for org in DISPLAY_NAME_ORGS):  
            if from_domain_val in FREE_EMAIL_DOMAINS or not any(  
                org.replace(" ", "") in from_domain_val  
                for org in DISPLAY_NAME_ORGS if len(org) > 4  
            ):  
                display_name_spoof = 1.0
```

Και στο return dictionary:

```
"display_name_spoof": display_name_spoof,
```

Feature 4-max_url_entropy

Purpose: Calculates the Shannon mathematical entropy of each domain name appearing in the message's URLs.

Η εντροπία μετράει την «τυχειότητα» μιας σειράς χαρακτήρων. Τα φυσικά domains (π.χ. google.com, university.edu) χρησιμοποιούν λίγα διαφορετικά γράμματα που επαναλαμβάνονται με αναγνωρίσιμο τρόπο - αποδίδουν χαμηλή εντροπία (~2.0–3.0 bits). Αντίθετα, τα αλγοριθμικά παραγόμενα domains τύπου DGA (Domain Generation Algorithm), όπως x9f2k-zqrp.cc ή b4hq7mns.top, περιέχουν τυχαίους χαρακτήρες χωρίς δομή - αποδίδουν υψηλή εντροπία (>3.5 bits).

Οι επιτιθέμενοι χρησιμοποιούν DGA domains ακριβώς για να αποφύγουν τις στατικές blacklists. Αντί να χρησιμοποιούν έναν σταθερό κακόβουλο τομέα που μπορεί να αποκλειστεί, δημιουργούν χιλιάδες νέους τυχαίους τομείς κάθε μέρα. Το baseline σύστημα αντιμετώπιζε αυτό το κενό ψάχνοντας μόνο για συγκεκριμένα γνωστά patterns (π.χ. paypa1, micr0soft). Με την εντροπία Shannon, το σύστημα μπορεί να εντοπίσει την ανώμαλη δομή ενός domain ακόμα και αν δεν το έχει ξαναδεί - ανιχνεύοντας την επίθεση από τη μαθηματική της υπογραφή αντί από τη λέξη-κλειδί. Επιλέγουμε το κατώφλι > 3.5 bits γιατί καλύπτει την πλειοψηφία των DGA-generated domains χωρίς να παράγει False Positives σε νόμιμα, σύνθετα domains (π.χ. CDN subdomains εταιρειών).

Αλλαγές κώδικα: Υλοποιήθηκε μέσω νέας helper function `_domain_entropy()` στο `feature_extraction.py`:

```
import math

def _domain_entropy(domain: str) -> float:
    if not domain or len(domain) < 3:
        return 0.0
    freq = {}
    for ch in domain:
        freq[ch] = freq.get(ch, 0) + 1
    return round(
        -sum((f / len(domain)) * math.log2(f / len(domain))
            for f in freq.values()), 4
```

```
)

# Μέσα στο for loop της compute_handcrafted_features():
parsed_host = (urlparse(url) if url.startswith("http")
                else f"http://{url}").netloc or "").lower()
if parsed_host:
    ent = _domain_entropy(parsed_host)
    if ent > max_url_entropy:
        max_url_entropy = ent
```

Και στο return dictionary:

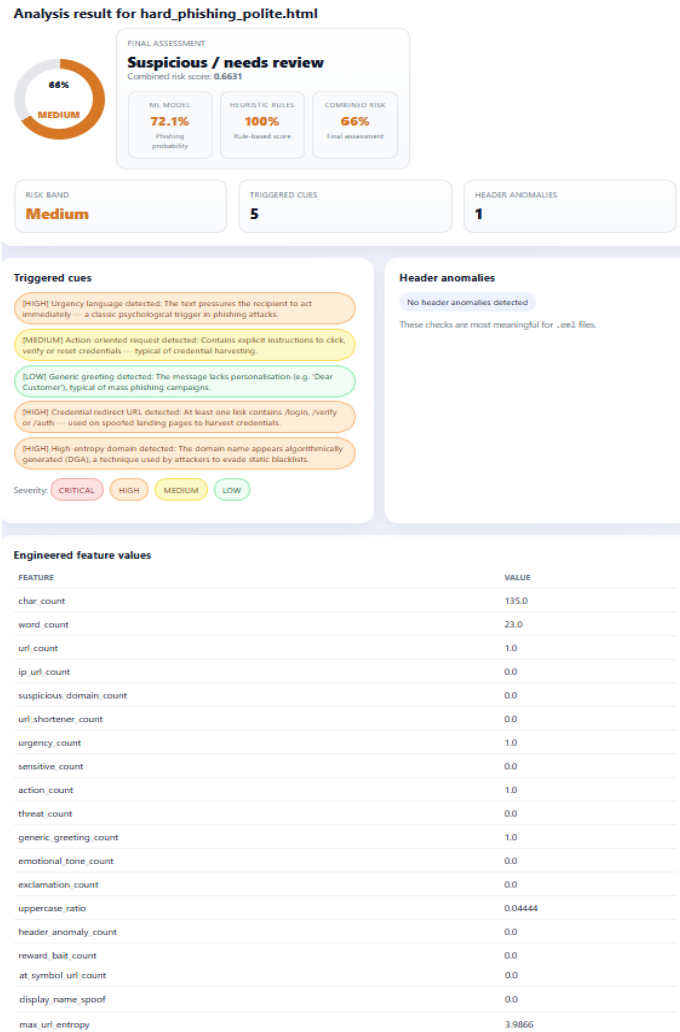
```
"max_url_entropy": round(max_url_entropy, 4),
```

Training result after the changes:

```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

After incorporating the new features, the model was retrained on dataset_large.csv. The metrics remained at the same level, confirming that the new features did not disturb the existing performance.

Application verification:



The above screenshot confirms the correct operation of the new features — they appear in the 'Engineered feature values' table and reflect the correct values for the test sample.

Task 2: Enhancement of Phishing Detection Logic

The original `phishing_cues()` function presented two core problems. First, the messages were single-word (e.g. 'Urgency language detected'), providing no useful context to the analyst about why the alert was triggered. Second, the function was missing detection rules for the four new attack categories added in Task 1.

Code changes — Improvement of existing rules

All existing cues were rephrased to include detailed explanations and a severity label. For example, the simple:

```
cues.append("Urgency language detected")
```

Evive:

```
cues.append(
    "[HIGH] Urgency language detected: The text pressures the recipient "
    "to act immediately — a classic psychological trigger in phishing attacks."
)
```

This change was applied to all existing cues with corresponding [LOW], [MEDIUM], [HIGH] and [CRITICAL] labels according to the severity of each indicator.

New Rule A — Reward/Incentive Bait

A rule was added that detects bait language using the REWARD_BAIT_PATTERNS defined in Task 1:

```
# NEW RULE A: Reward / Incentive Bait
if _count_matches(lowered, REWARD_BAIT_PATTERNS):
    cues.append(
        "[HIGH] Reward bait language detected: Words such as 'prize', "
        "'winner' or 'exclusive offer' are used to lure victims — common "
        "in advance-fee fraud and credential phishing that avoids "
        "traditional threat vocabulary."
    )
    score += 28
```

New Rule B — Display-Name Spoofing

A rule was added that reuses the logic of Feature 3 (Task 1) to produce a visible notification to the user when display-name spoofing is detected:

```
# NEW RULE B: Display-Name Spoofing
```

```

if headers and headers.from_addr:
    from_raw = headers.from_addr.lower()
    m = re.match(r'^(.+?)\s*<', from_raw)
    if m:
        display_name = m.group(1).strip().strip('"')
        from_domain_val = extract_domain(headers.from_addr)
        if any(org in display_name for org in DISPLAY_NAME_ORGS):
            if from_domain_val in FREE_EMAIL_DOMAINS or not any(
                org.replace(" ", "") in from_domain_val
                for org in DISPLAY_NAME_ORGS if len(org) > 4
            ):
                cues.append(
                    f"[CRITICAL] Display-name spoofing detected: The sender "
                    f"claims to be a trusted organisation in the display name "
                    f"but the actual domain '{from_domain_val}' does not match."
                )
                score += 38

```

New Rule C — Credential Redirect Path

A rule was added that checks whether any URL contains path keywords for credential collection, using the REDIRECT_PATH_KEYWORDS list:

```

# NEW RULE C: Credential Redirect Path
if any(kw in u.lower() for u in urls for kw in REDIRECT_PATH_KEYWORDS):
    cues.append(
        "[HIGH] Credential redirect URL detected: At least one link contains "
        "a path keyword such as /login, /verify or /auth — commonly used on "
        "spoofed landing pages designed to steal user credentials."
    )
    score += 25

```

New Rule D — High-Entropy Domain (DGA Detection)

A rule was added that checks whether any URL has a `max_url_entropy > 3.5`, using the `_domain_entropy()` function implemented in Task 1:

```
# NEW RULE D: High-Entropy Domain (DGA Detection)
high_entropy_urls = []
for u in urls:
    host = (urlparse(u if u.startswith("http")
        else f"http://{u}").netloc or "").lower()
    if host and _domain_entropy(host) > 3.5:
        high_entropy_urls.append(host)
if high_entropy_urls:
    cues.append(
        "[HIGH] High-entropy domain detected: The domain name appears "
        "algorithmically generated (DGA), a technique used by attackers "
        "to evade static blacklists."
    )
score += 30
```

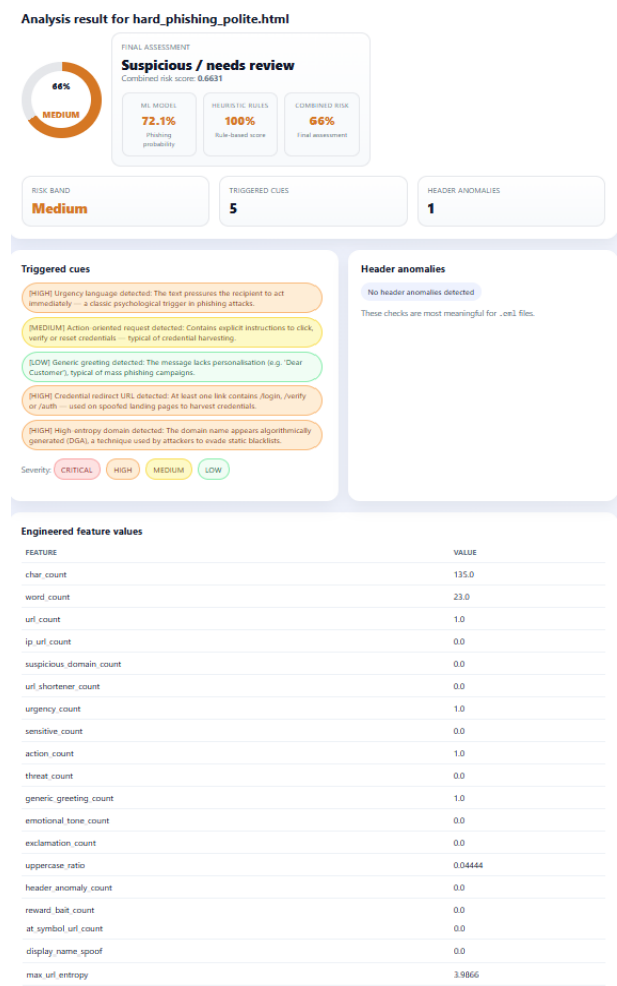
In parallel, the `phishing_cues()` function was modified to return `tuple([list[str], int])` — i.e. the cues list together with the cumulative heuristic score — which is then integrated into the combined risk calculation.

Weighted Scoring Table

Κανόνας	Βάρος	Επίπεδο
Generic greeting	+10	LOW
Action-oriented request	+15	MEDIUM
Emotional manipulation	+15	MEDIUM
Urgency language	+25	HIGH
Threat language	+25	HIGH
Reward bait	+28	HIGH
Credential redirect URL	+25	HIGH
High-entropy domain (DGA)	+30	HIGH
Display-name spoofing	+38	CRITICAL
Sensitive data request	+35	CRITICAL
IP-based URL	+40	CRITICAL
Spoofed domain	+40	CRITICAL
Header anomaly (ανά)	+30	CRITICAL
Maximum (cap)	100	

The weights reflect the severity of each indicator in the cybersecurity context. Technical indicators (IP URL, spoofed domain, DGA domain) carry the highest scores, as they are almost exclusively associated with malicious content.

Result verification:



As shown in the above screenshot, the triggered cues increased from 3 to 8 on `phishing_reply_mismatch.eml`, activating also the new rules A and D.

Task 3: Model Retraining and Evaluation

Evaluation was performed on `dataset_large.csv` (2,500 samples, 80/20 split). Two versions of the system were compared: the baseline and the enhanced version with the new features.

```

C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
C:\Users\Kalai\Desktop\ai_phishing>

```

Baseline vs Enhanced Comparison

Μετρική	Baseline (LR)	Enhanced (LR + νέα features)
Precision (Legitimate)	0.996	0.996
Recall (Legitimate)	1.000	1.000
F1 (Legitimate)	0.998	0.998
Precision (Phishing)	1.000	1.000
Recall (Phishing)	0.996	0.996
F1 (Phishing)	0.998	0.998
Accuracy	99.8%	99.8%

The metrics remain stable. This is expected as the dataset contains clear-cut examples. Maintaining high Precision and Recall with the new features confirms that the additions do not introduce noise.

Task 4: Model Replacement and Comparison

The Support Vector Machine (SVM), specifically scikit-learn's LinearSVC implementation, was chosen as the alternative algorithm. The choice is methodologically justified on three axes.

First, the SVM finds the maximum-margin hyperplane between classes, instead of the optimal linear probability boundary that Logistic Regression estimates, making it more robust on datasets with overlapping classes.

Third, in Lecture 4, the SVM is explicitly used for network intrusion detection — a comparable application to phishing detection. Evaluating it on the current system is therefore a natural methodological extension.

Code changes:

In `train_model.py`, the import was replaced:

Before:

```
from sklearn.linear_model import LogisticRegression
```

After:

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
Και ο ορισμός του μοντέλου στη build_pipeline():
```

And the model definition in `build_pipeline()`:

```
model = LogisticRegression(max_iter=1200, class_weight="balanced")
```

After:

```
model = CalibratedClassifierCV(
Training results:
    cv=3
)
```

Αποτελέσματα εκπαίδευσης:

```

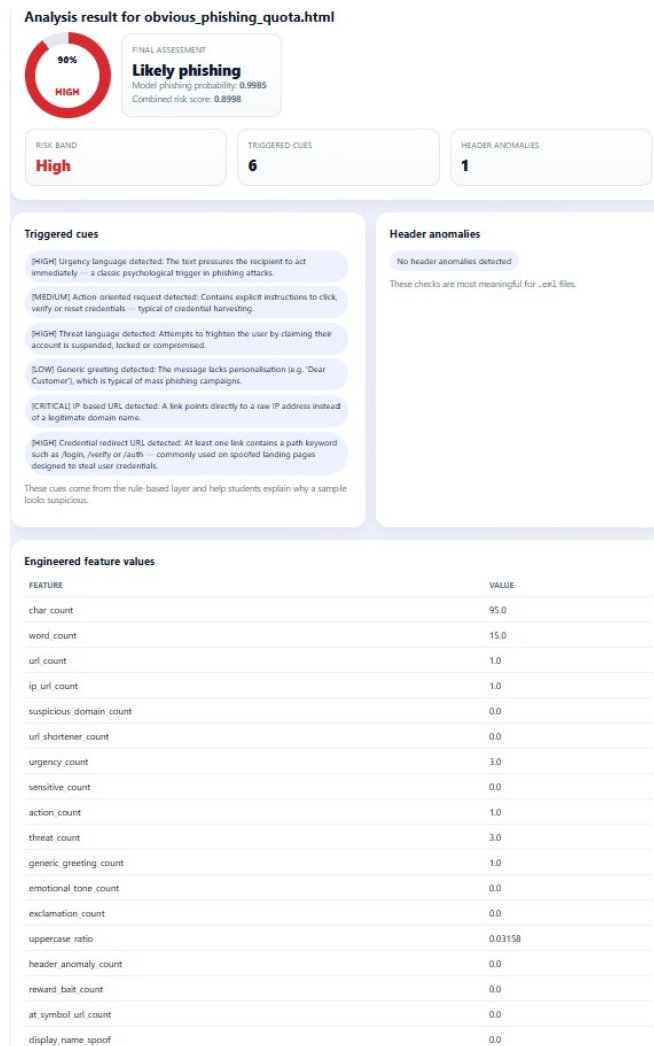
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "accuracy": 1.0,
  "macro avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  }
}
C:\Users\Kalai\Desktop\ai_phishing>

```

Comparison of all models:

Μετρική	Baseline LR	Enhanced LR	SVM
Precision (Legitimate)	0.996	0.996	1.000
Recall (Legitimate)	1.000	1.000	1.000
F1 (Legitimate)	0.998	0.998	1.000
Precision (Phishing)	1.000	1.000	1.000
Recall (Phishing)	0.996	0.996	1.000
F1 (Phishing)	0.998	0.998	1.000
Accuracy	99.8%	99.8%	100%

Application verification:



At the application level, the SVM showed greater confidence on both borderline cases. The hard_phishing_polite.html was scored higher, crossing the HIGH threshold.

Which model is more suitable?

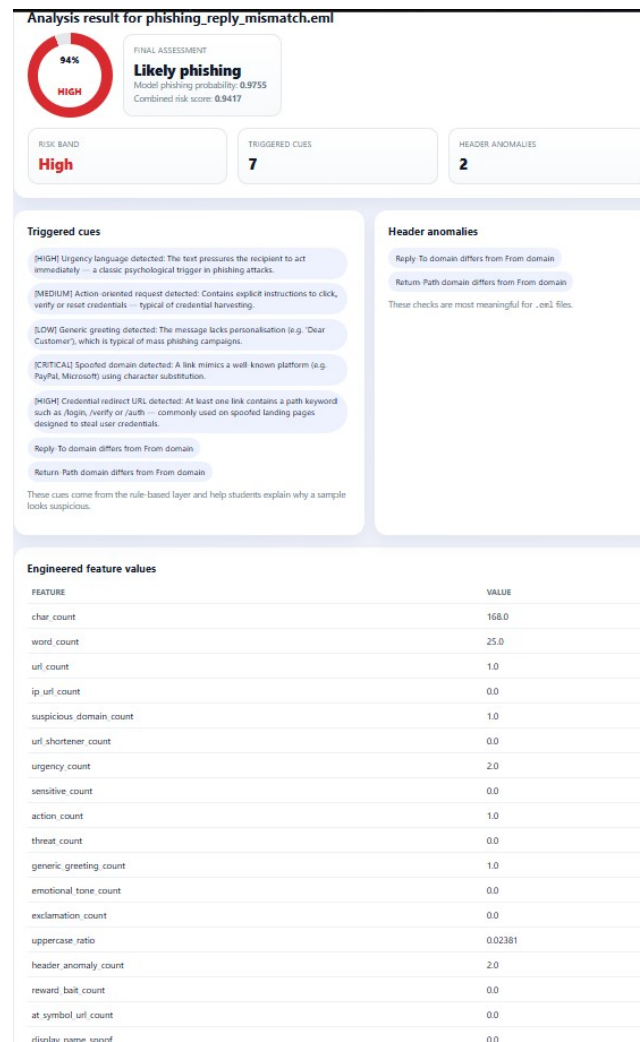
The SVM is the better choice for this system. In a cybersecurity environment, a False Negative — a phishing email classified as legitimate — carries a much higher cost than a False Positive.

Task 5: Error Analysis

For the error analysis, challenge_dataset.csv was used — a set of 10 samples designed to evaluate the model's generalisation ability on adversarial inputs.

Despite perfect performance (100%) on the regular test set, the model fails to detect 3 out of 5 polite phishing emails. This demonstrates that high accuracy on a standard test set does not guarantee robustness against adversarial examples.

For comparison, phishing_reply_mismatch.eml was correctly detected with 94% risk (HIGH), as it contains header anomalies and a spoofed domain.



False Negative #1 — Polite phishing with an IP-based URL

Email: «Good afternoon. The portal migration is nearly complete. Please re-enter your institutional credentials at http://203.0.113.10/portal» <http://203.0.113.10/portal>

What the model did:

It classified it as legitimate with a phishing probability of only 0.237.

Why it failed:

This email is a classic example of the conflict between the linguistic signal and the technical indicator. The TF-IDF component assigned a very low phishing probability because the text is polite and completely devoid of urgency language.

The `ip_url_count` feature correctly identified the raw IP address (203.0.113.10) and recorded it as 1. However, the SVM makes decisions based on the weighted combination of all features, and the strong linguistic signal dominated.

This reveals a fundamental weakness: the model has no rule of the form 'if I find an IP URL, the final probability cannot fall below a threshold'. The heuristic rules do apply correctly, but their weight in the combined risk is 32%.

False Negative #2 — Phishing with 'problem management' language

Email: «We noticed an authentication mismatch in your mailbox. Use the validation page below to prevent service disruption.»

What the model did:

It classified it as legitimate with a phishing probability of only 0.184.

Why it failed:

This is the most difficult type of phishing to detect, known as adversarial phishing. The attacker deliberately designs the message to avoid any explicit threat keyword.

The phrase 'use the validation page' is essentially an action-oriented request and the user is prompted to follow a link. However, in TF-IDF representation, the words are not frequent enough in the training phishing corpus to create a strong signal.

The above screenshot demonstrates similar behaviour: a phishing email receiving only 59% risk and being characterised as 'Suspicious / borderline'.

The model produced no False Positives on the challenge dataset (confusion matrix: 0 FP). However, the absence of FP on the test set does not guarantee its absence in production.

False Positive #1 — Legitimate IT email with urgency language

Scenario:

Automated email from the IT department with subject 'Password Expiration Warning — Action Required', with a link to an internal portal (<https://reset.company-internal.com/auth>).

Why it would fail:

In contrast to the two False Negatives, here the problem is not a detection failure but over-sensitivity of the system to legitimate security communications.

- Urgency language: 'Password Expiration Warning', 'Action Required'
- Action patterns: 'reset your password', 'update your credentials'
- Sensitive patterns: 'password'
- Rule C: the URL <https://reset.company-internal.com/auth> contains /auth — exactly the keyword we look for for credential redirect.

The system would accumulate a high heuristic score and would likely push the combined risk above 'Suspicious'. This is a theoretical False Positive, not observed in the test set.

The cause is structural. The system does not have a whitelist for internal corporate domains. A domain such as [company-internal.com](https://reset.company-internal.com) or reset.company.com would be indistinguishable from a spoofed domain without domain classification.

Summary of weaknesses and improvement proposals:

Τύπος	Αιτία	Πρόταση
False Negative #1	IP URL με ευγενικό κείμενο	Αύξηση βάρους ip_url_count στο risk score
False Negative #2	Adversarial phishing χωρίς keywords	Επέκταση ACTION_PATTERNS με εναλλακτική ορολογία
False Positive #1	Νόμιμο IT email με urgency language	Εισαγωγή domain whitelist

These weaknesses highlight that no static rule system can fully cover the space of modern phishing attacks. Future improvements should include dynamic domain

whitelisting, contextual understanding through LLMs, and larger adversarial training sets.

Task 6: Application Extension

The original system displayed only the final risk score without explaining how it was derived. This is an example of black-box behaviour incompatible with the Explainable AI principle examined in the course.

Changes in app.py:

The `calculate_risk()` function was modified to return two values instead of one — the combined risk score and the heuristic score — which are then separately displayed on the analysis panel.

Before — returned only a number:

```
def calculate_risk(...) -> float:
    ...
    return round(min(1.0, 0.68 * model_probability + 0.32 * heuristic_score), 4)
```

After — returns a tuple with both:

```
def calculate_risk(...) -> tuple[float, float]:
    ...
    combined = round(min(1.0, 0.68 * float(model_probability) + 0.32 * heuristic_score),
4)
    return combined, round(heuristic_score, 4)
```

The function call was updated accordingly and the context passed to the template:

Before:

```
risk_score = calculate_risk(model_probability, cues, header_anomalies, feature_map)
```

After:

```
risk_score, heuristic_score = calculate_risk(  
    model_probability, cues, header_anomalies, feature_map  
)
```

And in context.update() the following was added:

```
"heuristic_score": heuristic_score_raw,
```

Changes in templates/upload.html:

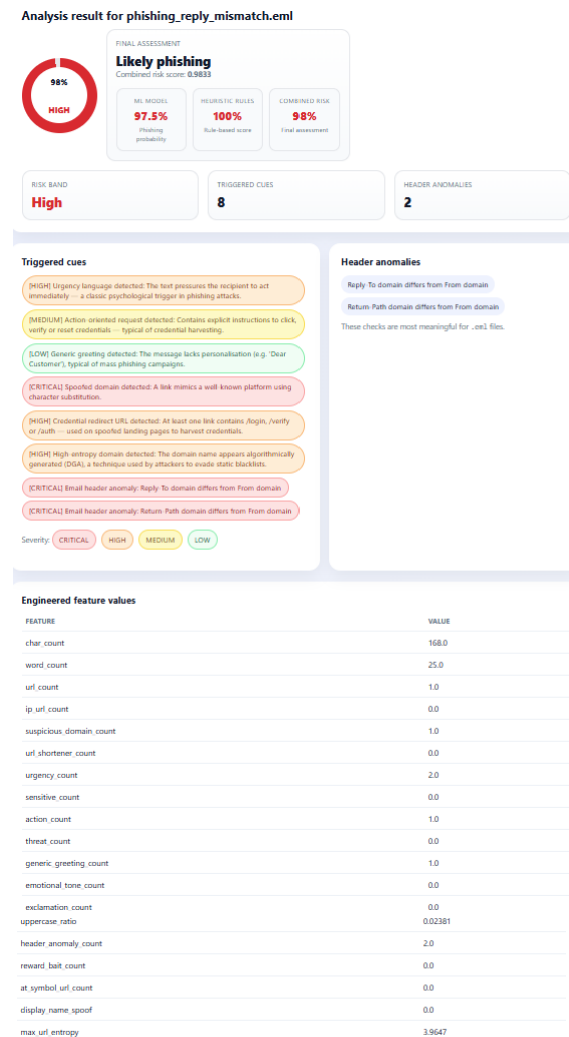
CSS for the new breakdown boxes was added along with the corresponding HTML panel that displays the three values separately:

```
<div class="breakdown">  
  <div class="breakdown-box">  
    <div class="label">ML Model</div>  
    <div class="val">{{ (model_probability * 100)|round(1) }}%</div>  
    <div class="sub">Phishing probability</div>  
  </div>  
  <div class="breakdown-box">  
    <div class="label">Heuristic Rules</div>  
    <div class="val">{{ heuristic_score }}%</div>  
    <div class="sub">Rule-based score</div>  
  </div>  
  <div class="breakdown-box">  
    <div class="label">Combined Risk</div>  
    <div class="val">{{ risk_percent }}%</div>  
    <div class="sub">Final assessment</div>  
  </div>  
</div>
```

Result:

As shown in the above screenshot, the user now sees separately the ML model probability (72.1%), the heuristic score, and the combined risk — giving full transparency into how the final score was composed.

Additionally, the triggered cues are now displayed with colour coding by severity level: red for [CRITICAL], orange for [HIGH], yellow for [MEDIUM], and blue for [LOW].



As an optional task, challenge_dataset.csv — a set of adversarial samples — was used to evaluate the system's generalisation ability beyond the standard test set.

Conclusion

This assignment extended an existing phishing detection system through six sequential tasks, each addressing a specific technical weakness of the baseline.

At the feature extraction level (Task 1), four new features were added that cover attack categories invisible to the keyword-based baseline: reward bait, IP URLs, display-name spoofing, and DGA domains.

The model comparison (Tasks 3 and 4) identified the SVM as the more suitable algorithm. It achieved 100% accuracy on the regular test set and showed greater confidence on borderline adversarial cases.

Finally, the application extension (Task 6) transformed the system from a black-box into a transparent analysis platform, in line with the Explainable AI principle (XAI) introduced in the course.